



Prem Predictor

1. Introduction

PremPredictor is a personal web-based project developed to create a structured and interactive way of predicting Premier League table outcomes. The idea was inspired by a strong interest in football and the competitive nature of making league predictions within social groups, where results are often tracked informally and disputes about who predicted what are settled from memory. The project aims to formalise this experience by providing a platform for creating private leagues, submitting predictions that cannot be seen or altered before a deadline, and comparing results once that deadline passes.

The application began as a single-page, offline tool in which all data lived in the browser. It has since been rebuilt as a multiplayer system in which several people, on separate devices, compete in the same league. That change moved the central problem from presentation to trust: if predictions are shared, the application must be able to guarantee that nobody has read or edited theirs early. The project therefore demonstrates not only client-side design and data handling, but the use of server-enforced authorisation rules as the mechanism by which a competitive guarantee is made real rather than merely displayed.

1.5 Project Function and What It Does

PremPredictor is a browser-based application that allows a group of people to compete on predicting the final Premier League table. One user creates a league and sets a deadline; others join it by following an invite link or typing a six-character code. Each player signs in, creates a short profile, and submits a full ranking of all twenty clubs.

Every prediction remains sealed until the deadline. No player, including the person who created the league, can read another player's table before that point. When the deadline passes the league opens for everyone at once, tables become visible, and the organiser can publish scores calculated against the real Premier League standings.

From a user's perspective the workflow is short. A league is created and shared, each player ranks twenty clubs by dragging, tapping, or typing, and the table is saved. Players can revise their entry as often as they like until the league closes. Once results are published the application ranks everyone, reveals the standings one position at a time starting from the bottom, and offers a downloadable PDF report comparing a player's prediction against the actual table.

2. Tech Stack Used

- **HTML5**
- **JavaScript (vanilla, ES modules; no front-end framework)**
- **Tailwind CSS v4, compiled at build time**
- **esbuild (bundling and code splitting)**
- **Firebase Authentication (Google sign-in)**
- **Cloud Firestore (database and security rules)**
- **Firebase Hosting**
- **Firebase App Check with reCAPTCHA v3**
- **jsPDF (bundled and loaded on demand)**
- **Plus Jakarta Sans, self-hosted as WOFF2**
- **football-data.org API (accessed from CI only, never from the browser)**
- **GitHub Actions (scheduled daily job)**
- **Browser localStorage (local convenience cache only)**

.HTML provides the document shell. JavaScript handles routing, rendering, validation, and scoring, organised as ES modules with a small view layer rather than a framework. Tailwind supplies the styling system, with a custom component layer for the parts Tailwind does not express well, such as the prediction board and the navigation shell.

Several choices in this list are consequences of one decision: the application enforces a strict Content-Security-Policy of default-src 'none', with font-src 'self' and no inline scripts. Nothing may be loaded from a CDN. Tailwind, jsPDF, and the typeface are therefore compiled or bundled into the application's own origin rather than fetched at runtime, which is why the earlier CDN-based versions of all three were replaced.

Firestore replaced localStorage as the source of truth. localStorage still exists, but only as a per-browser convenience: the authoritative record of which leagues an account belongs to is stored against the account, so signing in on a second device produces the same leagues and the same saved predictions.

3. Architecture Used

The application is a static single-page site with no server of its own. It is bundled by esbuild into an entry chunk plus a lazily loaded chunk containing the PDF machinery, and served by Firebase Hosting. Routing is hash-based, so every page has a real URL, the browser's back and forward buttons behave correctly, and an invite is simply a link.

The absence of a server does not mean the absence of server-side logic. Firestore security rules are evaluated by Firebase on every read and write, and they are where the application's guarantees live. The client is treated as untrusted throughout: hiding a control in the interface is a courtesy, and the rule that refuses the corresponding write is the actual protection.

Data model:

- `leagues/{leagueld}` — name, owner, deadline, and the switches describing the league's state
- `leagues/{leagueld}/members/{uid}` — the public roster: display name and picture, no prediction data, so it can be read before the deadline
- `leagues/{leagueld}/entries/{uid}` — the twenty-club prediction, unreadable by anyone else until the league reveals
- `users/{uid}` — display name and picture, deliberately holding no email address
- `users/{uid}/leagues/{leagueld}` — a private index of the leagues an account belongs to, readable only by that account
- `joinCodes/{code}` — a short typeable alias pointing at a league id

Source layout:

- `index.html` — document shell, live regions, and mount points
- `app.js` — entry point: boots Firebase and keeps the page in sync with the URL and auth state
- `src/state.js` — all Firebase access; views never touch Firestore directly
- `src/clubs.js` — the canonical club list, deliberately dependency-free so build tooling can read it
- `src/router.js`, `src/ui.js`, `src/dnd.js` — routing, presentation primitives, and drag-and-drop
- `src/views/*.js` — one module per page
- `src/table-data.js` — the live Premier League standings and their fallback
- `report.js` — PDF generation, imported on demand
- `firestore.rules` — the authorisation rules
- `scripts/fetch-table.mjs` and `.github/workflows/update-table.yml` — the daily standings job
- `tests/rules.test.mjs` — 66 tests covering the security rules
- `build.mjs` — the bundler, which also refuses to build an invalid club list

A league id is a 16-character random string and functions as a capability: holding it is what grants access, which is what makes a plain link a working invitation. The rules permit fetching a league by id but never permit listing the collection, so leagues cannot be enumerated by anyone who has not been given a link.

4. Full Functionality with Technical Implementation

On load the application restores any existing session before deciding what to render, so a returning user does not see the signed-out page flash first. Authentication is Google sign-in only. Email sign-in links were implemented and later removed: Firebase's built-in mail has a small per-project daily allowance which a single afternoon of testing exhausts, and the alternatives that avoid an email address entirely either cost money or require a server holding a signing key. The

application receives an email address from Google in the sign-in token and never writes it to the database; the only personal data stored is a display name and a picture, both chosen by the user.

Every rule requires a real, verified account. Anonymous sessions are refused server-side, so a league cannot be entered repeatedly under invented names.

League membership is recorded against the account rather than the browser, which is what allows predictions to follow a user between devices. Leagues are joined by link or by a six-character code stored as its own document keyed by the code, so resolving a code is a direct lookup rather than a query — the leagues collection remains unlistable.

Predictions are entered on a board of twenty fixed positions. A club may be dragged to any position, tapped to fill the lowest empty one, or typed into a slot with prefix matching. On a phone the full board does not fit, so the application defaults to a one-position-at-a-time mode sized exactly to the viewport, in which only the club list scrolls and the save control is always reachable. Drag-and-drop is implemented with Pointer Events and a movement threshold that distinguishes a tap from a drag, and is fully operable from the keyboard: a position can be lifted with the space bar, moved with the arrow keys, and dropped. Interface changes are announced through ARIA live regions, dialogs trap focus and restore it on close, and animation is suppressed when the operating system requests reduced motion.

The sealed-until-deadline guarantee is enforced entirely in the security rules. Reading another player's entry is refused outright before the league reveals; writing to an entry is refused once it is closed. Implementing the read side revealed a subtlety in how Firestore evaluates rules: on a query, `request.time` is pinned to the read snapshot rather than to wall-clock time, so a rule of the form "readable once the deadline has passed" never begins to pass, and predictions would have stayed sealed permanently. Read visibility therefore keys off a stored flag which any member may flip once the server agrees the deadline has passed — checked on the write, where `request.time` is the request's own time. Whoever has the page open at the deadline trips it for everybody, so the reveal does not depend on the organiser being present.

A league occupies one of four states, presented consistently across every page with its own colour: open for predictions, locked, revealed, and results published. Several are true at once for most of a league's life, so the most recently reached state is the one displayed. The organiser can close a league early, hold it open past its deadline, reveal tables, and publish scores, and every one of those switches is reversible — an earlier version made the deadline final, which proved wrong for informal groups where a straggler needs letting in.

Scoring compares a prediction against the real table. A club in exactly the right position scores three points, one place out scores two, and two places out scores one. Bonus points are added for correctly naming the champion, for each club correctly placed in the top four, and for each club correctly placed in the relegation zone. A perfect table scores 77. Scores are recalculated from the current standings rather than stored, so they stay correct as results come in.

The real standings are supplied by football-data.org. Because the application is a static site with no server, the API is called from a GitHub Actions job on a single daily schedule at midnight; the job writes a static file, commits it, and redeploys. The API token is held in CI and never reaches a browser, which is also what allows the Content-Security-Policy to keep `connect-src 'self'`. The once-a-day limit is enforced rather than merely documented: the workflow has one schedule, the script refuses to call the API if the file was written less than twenty hours ago, and no code path in the application can reach the API at all. Before any matches are played the feed reports every club in position one on zero points, which is accurate but is not a table; the application detects this and lists the clubs alphabetically without positions rather than presenting an alphabetical accident as a ranking.

Two documents can be generated in the browser with jsPDF, which is code-split and fetched only when requested. Before results are published a player can download a prediction sheet: a single full-page table of their twenty clubs with qualification and relegation places marked. Once scores are published the full report becomes available, comparing predicted against actual positions with a points breakdown and the player's best and worst calls. Predictions can also be exported and imported as JSON, or copied from another league the player is in. Club crests are downscaled before being embedded, which reduced a report from roughly 53 MB to under 1 MB.

Club names are load-bearing, because they are also crest filenames. A name that does not match its file renders a broken image everywhere a club appears and, because the same list decides which clubs the application recognises, silently causes live standings to be rejected for scoring. This occurred twice from a stray "FC" suffix, so the club list was moved into a dependency-free module that Node can import, and both the build and the standings job now validate against it. The build exits with an error rather than producing a bundle in which a club has no crest.

Security beyond the rules is handled by App Check with reCAPTCHA v3, which attests that requests originate from the genuine application rather than a script written against the project, and by the Content-Security-Policy described above. The security rules are covered by 66 automated tests run against the Firestore emulator, asserting both what is permitted and what is refused, since a rule that is too permissive fails silently in a way ordinary testing does not catch.

Overall the project demonstrates a complete multiplayer system built without a server of its own: authentication, an authorisation model expressed as executable rules, a shared database, a scheduled data pipeline, client-side document generation, and an accessible interface, deployed as a static site.